Software Engineering Department Braude College of Engineering
Final Project – Phase B (61999)

Cross-Domain Transfer Learning: From Tropical Cyclones to Insect Bite Classification

Bug bite classification using a CNN based TC classification model

CODE: 26-1-R-26

By:
Ishay Yulzary
Natan Trostianitser

Advisors:
Mrs. Elena Kramer
Dr. Dan Lemberg

Link to GitHub:

https://github.com/idshay16/Bug-bite-classification-using-a-TC-classification-model

Contents

1 Abstract

Abstract

Diagnosing the specific cause of an insect bite remains a largely manual and subjective clinical task, as medically distinct bite types – ranging from benign mosquito bites to necrotic spider bites – frequently present with visually similar early-stage symptoms. This project investigates whether an unconventional, structurally motivated transfer learning strategy can improve automated bite classification beyond the semantic transfer learning (e.g., ImageNet pretraining) used in prior work. We identify a morphological parallel between tropical cyclone satellite imagery – a concentric "eye" surrounded by radiating cloud bands – and insect bite lesions, such as the necrotic center of a spider bite or the expanding radial pattern of a tick bite, and hypothesize that features learned from cyclone classification transfer productively to this medical domain.

We implement a three-phase pipeline (ImageNet $\rightarrow$ cyclone $\rightarrow$ bug-bite), which we refer to as *Transfer-and-Calibrate* (TC), and compare it against a two-phase *control* pipeline (ImageNet $\rightarrow$ bug-bite) across three backbone architectures (ConvNeXt-Tiny, DenseNet-121, InceptionV3), five bite categories (ants, bed bugs, ticks/fleas, mosquitoes, spiders), and a hyperparameter grid of 7 seeds $\times$ 3 label-smoothing values $\times$ 3 learning rates, totaling 378 trained models. Across 189 runs per condition, TC achieves higher mean validation accuracy, macro F1-score, precision, and recall than the control baseline, along with lower final validation loss, and is the top-performing configuration for every one of the five bite categories. While control attains a marginally higher peak accuracy in isolated runs, its performance is highly sensitive to the specific hyperparameter draw, whereas TC exhibits substantially more consistent behavior across the grid – a property we consider important for deployment settings where hyperparameters cannot be hand-picked in advance. We further integrate Grad-CAM and SHAP-based explainability to verify that model predictions are grounded in medically meaningful lesion features, and use these tools to characterize a persistent Spider/Mosquito confusion as a genuine visual-overlap failure mode rather than a training artifact. No single backbone dominates across all categories, motivating a category-aware ensembling strategy as a direction for future work. Overall, our findings support cyclone imagery as a viable, and unexpectedly effective, source domain for cross-domain transfer learning in dermatological bite classification.

2 Introduction and background

Diagnosing the specific cause of an insect bite is a task that remains largely manual and subjective in clinical practice, despite the wide range of severity such bites can present, from minor irritation to necrotic tissue damage and disease transmission. In this section we review the clinical motivation for automating bite classification, the theoretical foundations of the techniques we used, and the specific research gap our project addresses through its cross-domain transfer learning approach.

2.1 Clinical Motivation

Bites and stings inflicted by arthropods, a category that includes insects, spiders, and other jointed-leg organisms, represent a significant and universal health concern. While the majority of cases resolve without intervention, bites from certain vectors, particularly necrotic spiders or disease-carrying insects, can lead to rapid tissue death, systemic infection, or long-term scarring if not identified and treated promptly. The underlying medical risk arises from several distinct mechanisms: direct mechanical injury and the resulting risk of secondary bacterial infection, allergic reactions to salivary antigens that can range from mild itching to anaphylaxis, venom toxicity, and the transmission of disease by the arthropod acting as a biological vector [9].

The central diagnostic difficulty, as we understand it, is that these medically distinct causes frequently produce visually similar early-stage symptoms, typically redness, swelling, and localized inflammation, making it difficult for a physician to distinguish, for example, a benign mosquito bite from an early-stage tick bite or a dangerous spider bite based on visual inspection alone. In practice, this ambiguity often forces clinicians into a reactive "watch and wait" approach rather than an immediate targeted treatment such as antivenom or antibiotics, delaying appropriate care and placing unnecessary strain on medical resources. This is the gap that motivated us to look into an automated, image-based classification tool capable of identifying the specific insect vector responsible for a given bite.

2.2 Limitations of Conventional Approaches

Standard computer vision models, and by extension standard applications of transfer learning to dermatological image classification, are typically pretrained on large generic datasets composed of rigid, well-defined objects such as vehicles, animals, or household items. As Hosna et al. explain, transfer learning normally works by reusing a network trained on one task as the starting point for a related task, so that the target model does not need to learn everything from scratch [2]. Skin lesions, by contrast, present as amorphous regions with fading, poorly defined boundaries, a mismatch that limits how directly these generic pretrained features apply to our domain. Prior work on automated bite classification [4, 3], while establishing the general feasibility of deep learning for this task, has largely relied on this conventional strategy: initializing models with weights pretrained on large-scale general-purpose datasets such as ImageNet and subsequently fine-tuning on bite imagery. In this paradigm, a substantial portion of the model's training effort is spent suppressing visual features learned from irrelevant object categories in order to isolate the dermatological patterns relevant to the diagnostic task, a process that is both computationally inefficient and dependent on having a sufficiently large target dataset to overcome this mismatch.

2.3 A Cross-Domain Structural Parallel

Our approach departs from this conventional semantic transfer learning strategy by identifying a structural, rather than semantic, parallel between insect bite lesions and an entirely different visual domain: tropical cyclone imagery observed from satellites. A tropical cyclone is a rotating, low-pressure weather system characterized by a region of calm, low-intensity wind at its center, commonly referred to as the eye, surrounded by a ring of intense convective cloud activity, with spiral bands of cloud extending outward at greater radius [11]. This concentric structure, a distinct central feature surrounded by radiating, gradient-rich patterns, bears a notable morphological resemblance to certain insect bite presentations: the necrotic center characteristic of a spider bite parallels the calm eye of a storm, while the expanding radial pattern associated with tick bites (erythema migrans) parallels the radiating cloud bands of a cyclone.

Based on this observation, we hypothesized that a model trained to recognize the concentric, swirling structural patterns of cyclones, such as the Hybrid Cyclone Intensity Estimation Framework developed for analyzing storms in the North Indian Ocean [8], has learned feature extractors that are also relevant to detecting the analogous geometric anomalies present in insect bite lesions. Rather than relying on a generic source domain and expending training effort to filter out irrelevant semantic content, we wanted to reuse a source domain whose primary learned visual feature, the geometry of a concentric, gradient-rich circular anomaly, is already structurally aligned with our target task. This is consistent with the broader transfer learning literature: in their well-known study on feature transferability, Yosinski et al. show that the earlier layers of deep convolutional networks tend to learn generic, transferable visual primitives such as edges, curves, and gradients, while deeper layers encode increasingly task-specific semantic features, and that this generic-to-specific hierarchy can be exploited by fine-tuning a network on a new, related task [12]. We rely on essentially the same idea here, just applied across two domains (meteorology and dermatology) that are not normally considered related. Precedent for this kind of unconventional cross-domain transfer, for instance adapting networks originally designed for satellite road imagery to unrelated pattern recognition tasks [1], further supports the plausibility of our strategy.

2.4 Research Gap and Objective

The classification task we address in this project targets five representative bite categories, spanning a baseline benign case, clustered bite patterns, acute central punctures, expanding radial patterns, and necrotic-center presentations, corresponding respectively to mosquito bites, flea or bed bug bites, hymenoptera stings, tick bites, and spider bites. While prior automated bite classification systems [4, 3] have demonstrated the general viability of deep learning for this problem, to the best of our knowledge none have explored a geometric, cross-domain transfer strategy of the kind we propose here. Moreover, because our source architecture was originally developed for meteorological rather than medical data, adapting it to a diagnostic context raises an additional requirement for transparency: we need to verify that the model's decision-making is grounded in medically meaningful visual patterns rather than incidental artifacts inherited from the cyclone domain. This is why we chose to integrate explainable AI techniques alongside a multi-metric evaluation strategy, including precision, recall, F1-score and MCC (Matthews Correlation Coefficient), so that we could jointly assess both the accuracy and the interpretability of our resulting diagnostic system.

3 General Description

We designed our proposed system as an AI-based clinical decision-support tool intended to assist healthcare professionals in identifying the likely source of an insect bite or sting from a single photograph. Our primary objective was to reduce the time required for preliminary identification, thereby enabling more rapid and informed treatment decisions. The system is not designed to replace clinical judgment; rather, it functions as an auxiliary information source that supplements, rather than supersedes, the diagnostic process undertaken by a qualified healthcare professional. Our system employs a transfer learning approach, in which deep learning architectures originally developed for tropical cyclone intensity estimation are adapted for the task of insect bite classification. As mentioned above, this approach is motivated by the observation that satellite imagery of cyclonic systems and photographs of certain insect bite lesions share structurally similar visual characteristics – namely, radial and circular patterns – that are amenable to feature extraction by convolutional neural networks. We can summarize the system's operation in the following stages:

1. An image of the affected area is captured or uploaded by the user via a standard camera or smartphone device.
2. The input image undergoes preprocessing and is subsequently provided to an ensemble of three independently fine-tuned deep learning models.
3. Each constituent model generates an independent prediction, classifying the image into one of five predefined insect bite categories.
4. The individual model predictions are aggregated via a soft voting mechanism, in which the predicted class probabilities from each model are averaged, and the final ensemble classification is determined by the class with the highest mean probability.
5. The system outputs the predicted category alongside a Grad-CAM–based explainability visualization, which highlights the image regions that most strongly influenced the model's prediction.

We architected the system as a lightweight image classification pipeline requiring only a single photographic input, without reliance on specialized medical imaging equipment or manual feature engineering. This design supports its integration into time-constrained clinical environments. We designed the system with the following categories of healthcare professionals in mind as its intended user base:

1. Emergency department physicians and triage personnel, who require rapid assistance when evaluating patients presenting with insect bites of unknown etymology.
2. General practitioners and family physicians, who may benefit from an auxiliary decision-support tool during the course of routine clinical examinations.
3. Urgent-care clinicians, for whom expedited preliminary identification of insect bites can meaningfully inform treatment planning.

We want to emphasize that the system is intended exclusively as a decision-support instrument. Responsibility for final diagnosis and treatment determination remains with the attending healthcare professional at all times.

4 Solution Description

Our objective in this project was to develop a highly accurate classification system capable of identifying skin lesions resulting from five specific pests: ants, bed bugs, ticks/fleas, mosquitoes, and spiders. We designed each prediction produced by the system to be accompanied by an explicit confidence score, allowing the reliability of a given classification to be assessed alongside the predicted label. To improve performance beyond what standard architectures typically achieve, we employed a distinctive transfer learning strategy that exploits structural and visual similarities between meteorological cyclone imagery and insect bite lesions, using cyclone classification as an intermediate domain rather than training directly from a generic pretrained network to the target task – an approach that goes a step beyond the "friendly" beginner recipe for transfer learning described by Hosna et al. [2], since our source and target domains are not obviously related at the semantic level. We implemented the entire system in a Python and Jupyter Notebook environment.

The core deliverable, however, is not a single trained model but an automated experimentation framework that trains and validates this hypothesis at scale: 63 hyperparameter configurations (3 architectures $\times$ 3 label-smoothing values $\times$ 3 learning rates $\times$ 7 seeds), each comparing a control pipeline against the cyclone-intermediate pipeline, for 378 trained models in total. Given the multi-hour cost per configuration, we designed the framework to be resumable and crash-recoverable: configuration and checkpoint state persist to disk, an interrupted run resumes from the last completed checkpoint rather than restarting, and the hyperparameter grid can be extended without disturbing already-completed results.

4.1 Overall Architecture and Multi-Stage Learning Strategy

We built the system around a three-phase, progressively specialized learning strategy, rather than a conventional single-step fine-tuning process. In the first phase, we establish general knowledge by initializing the network with weights pretrained on a large-scale generic image dataset, giving the model foundational low- and mid-level visual recognition capabilities such as edge, texture, and shape detection. In the second phase, we perform domain refinement by training the network on the Indian Ocean cyclone dataset, adapting the model's learned representations to the complex, concentric visual patterns characteristic of cyclone imagery. We chose this intermediate stage because such concentric, radially symmetric structures bear a notable structural resemblance to the swelling and lesion patterns found in insect bites, making cyclone classification a useful proxy domain for bridging generic visual features and our target task. In the third phase, we specialize the model exclusively on insect bite diagnostics, adapting the representations refined during the cyclone stage to the five target pest categories.

This staged strategy results in an end-to-end pipeline encompassing automated training and evaluation, from raw image ingestion through final classification, without requiring manual intervention between stages once configured.

4.2 Classification Output and Confidence Estimation

While the core research framework evaluates the TC hypothesis in aggregate across the full hyperparameter sweep, we additionally built a standalone inference demo that showcases the trained models on individual images. Given an input image, the demo produces a predicted pest category among the five defined classes, together with an explicit confidence score derived from the model's output probability

distribution, letting a user distinguish high-confidence classifications from uncertain ones. This demo is a proof-of-concept illustration of what the framework's trained models can do, not a component of the research pipeline itself.

4.3 Explainable AI and Interpretability

We wanted to make sure the model's outputs were auditable and interpretable rather than functioning as an opaque black box, so we incorporated a comprehensive suite of Explainable AI diagnostics into our system. These diagnostics provide visual explanations of the image regions and features that drive each classification decision, letting us verify the correspondence between the model's predictions and physically meaningful lesion characteristics. We integrated this interpretability layer directly into the framework's evaluation pipeline: GradCAM, SHAP, and LIME produce a visual justification for individual classifications, while t-SNE and DiCE provide corroborating evidence at the embedding and cluster level across the validation set.

4.4 Success criteria for our project

- **Demonstrated Performance Improvement:** The TC pipeline (ImageNet→cyclone→bug-bite) achieves higher classification accuracy than the control pipeline (ImageNet→bug-bite), consistently across architectures and random seeds.
- **Optimal Hyperparameter Identification:** The hyperparameter sweep (label smoothing, learning rate) identifies, per architecture, the cyclone pre-training configuration that maximizes downstream bug-bite accuracy.
- **Interpretable, Corroborated Improvement:** Explainable AI diagnostics confirm that any accuracy gain from TC pre-training corresponds to attention on physically meaningful lesion features, rather than spurious correlations.

5 Research Process

To test whether our hypothesis was correct, we searched for models that could distinguish between different cyclones. We also had to figure out which bug species we wanted to be able to classify by their bites.

5.1 Initial Planning and Decision making

We began by trying to identify a suitable source model and datasets for the transfer learning process. We selected a project based on YOLOv3 detector and CNN based multiclass calssifer [10] that was capable of distinguishing between five types of tropical cyclones in the Indian Ocean [8]. Its architecture consisted of a binary classifier followed by a multi-class classifier and a regressor, giving us the components we needed for our transfer learning approach. Next, we identified an insect bite dataset from Kaggle containing five classes: Ants, Mosquitoes, Flea/Bed Bugs, Spiders, and Ticks. After obtaining both datasets, we noticed a morphological similarity between cyclone patterns and insect bite images, which is what motivated us to investigate whether knowledge learned from cyclone classification could be transferred to insect bite classification. We then went ahead and implemented the transfer learning pipeline.
The next stage involved evaluating the backbone models used by the multi-class classifier. The original implementation supported three convolutional neural network architectures: VGG-16, ResNet50, and InceptionV3. Since we felt these architectures were relatively old compared to more recent computer vision models, we did some additional research to identify stronger alternatives. Our goal was to evaluate several modern architectures and determine which achieved the best performance in the transfer learning task while maintaining high classification accuracy on the original cyclone dataset.

5.2 Development: Pre-training and models

To implement transfer learning in our project, we removed the unnecessary output heads of the original model – namely the binary classifier and the regressor – since our objective was solely multi-class classification of insect bite images. This modification simplified the architecture while preserving the pre-trained feature extraction backbone. As a result, the model could concentrate on learning the visual characteristics required for insect bite classification by leveraging the knowledge it had already acquired during cyclone classification.
We then evaluated the backbone models included in the original implementation without making any architectural modifications, just to see whether the transfer learning pipeline could be applied directly. The initial results were not great, since two of the three models achieved relatively low classification accuracy while exhibiting significant overfitting. We therefore decided to replace the two underperforming backbones with more recent architectures. To pick suitable replacements, we evaluated several candidate models, comparing both their classification accuracy and their tendency to overfit. Based on these experiments, we selected ConvNeXt-Tiny and DenseNet-121 while keeping InceptionV3 as our third backbone model. We chose these three models intentionally because they represent different convolutional neural network design philosophies, letting us not only compare their classification performance but also evaluate which architectural approach is most suitable for our transfer learning task. ConvNeXt-Tiny is a modern convolutional neural network that borrows several design principles from Vision Transformers while staying a fully convolutional architecture [7]. DenseNet-121 consists of four densely connected blocks separated by transition layers, promoting feature reuse and efficient gradient propagation. Incep-

tionV3 employs factorized and asymmetric convolutions to reduce computational cost while maintaining strong feature extraction capabilities. Using these three distinct architectures let us analyze their behavior within our transfer learning framework and identify the backbone that best suited insect bite classification. Furthermore, during training we found that careful hyperparameter tuning was necessary to achieve stable convergence and maximize performance. To make this process easier, we exposed several configurable parameters that let us systematically evaluate different training settings:

- **Label Smoothing** – reduces overconfidence in the model's predictions by assigning a small probability to non-target classes, thereby improving generalization and reducing overfitting.
- **Learning Rate** – controls the size of the optimization steps during training. Picking an appropriate learning rate balances convergence speed and training stability, preventing the optimizer from either overshooting the optimum or converging too slowly.
- **Random Seed** – ensures reproducibility of our experiments by fixing all sources of randomness. This let us fairly compare different backbone architectures and hyperparameter configurations while getting consistent and repeatable results.

5.3 Metrics and Explainability

Throughout the training process, we wanted a simple and consistent way to evaluate and compare the performance of our different backbone models and hyperparameter configurations. Since machine learning models can be assessed using a wide variety of evaluation metrics, we selected a set of complementary metrics that together give us a comprehensive view of classification performance:

- **Accuracy** – The proportion of correctly classified samples out of the total number of predictions.
- **Balanced Accuracy** – The average recall across all classes. We found this metric particularly suitable for our dataset, which exhibits class imbalance, as it gives equal importance to each class regardless of its size.
- **ROC-AUC** – Measures the model's ability to distinguish between classes across all possible classification thresholds. A higher ROC-AUC indicates better discriminative capability.
- **Cohen's Kappa** – Measures the agreement between the model's predictions and the ground truth while accounting for agreement that could occur by chance, giving us a more reliable evaluation than accuracy alone.
- **Matthews Correlation Coefficient (MCC)** – A robust metric that incorporates all four outcomes of the confusion matrix (true positives, true negatives, false positives, and false negatives). We found it particularly informative for our imbalanced dataset, where conventional accuracy could be misleading.

In addition to quantitative performance metrics, we incorporated Explainable Artificial Intelligence (XAI) techniques to better understand the decision-making process of our trained models. Rather than evaluating only the final prediction, we wanted these methods to give us insight into *why* the model reached a particular decision. Specifically, we employed the following XAI techniques:

- **SHAP** – to measure the contribution of each image region to the model's prediction using Shapley values.
- **LIME** – to generate local explanations by approximating the model's behavior around an individual prediction with an interpretable surrogate model.
- **DiCE** – to generate counterfactual explanations, illustrating the minimal changes required for an image to be classified as a different insect bite category.
- **Grad-CAM** – to visualize the regions of an image that most strongly influenced the model's prediction, letting us verify whether the network focused on medically relevant features rather than background artifacts.

By combining conventional evaluation metrics with XAI techniques, we were able to assess not only the predictive performance of our models but also the reliability and interpretability of their decision-making process.

6 Tools and Technologies

We developed the project using **Python 3** as our main programming language due to its extensive ecosystem of machine learning and data science libraries. We implemented the deep learning models using **PyTorch**, which gave us the framework for building, training, and evaluating neural networks. We used **torchvision** for image preprocessing and computer vision utilities, and **timm** to integrate modern pretrained architectures such as ConvNeXt-Tiny [7]. We performed image processing and manipulation using **OpenCV** and **Pillow (PIL)**.

For data processing and analysis, we used **NumPy**, **pandas**, and **scikit-learn**. These libraries supported our numerical computations, dataset management, preprocessing operations, evaluation metrics, dimensionality reduction, and clustering analysis. For visualization purposes, we used **Matplotlib** to generate training curves, confusion matrices, and performance comparisons. We also used **t-SNE** and silhouette scoring, implemented through **scikit-learn**, to analyze the learned feature representations and evaluate the separation between different classes.

To improve the interpretability of our model's decisions, we incorporated Explainable Artificial Intelligence (XAI) techniques. These XAI methods let us understand the reasoning behind our model's predictions by identifying the image regions and features that influenced the classification results. We implemented **Grad-CAM**, **SHAP**, **LIME**, and **DiCE**. For our data acquisition and preprocessing pipeline, we used **requests** for automated data retrieval, **netCDF4** for processing meteorological data stored in NetCDF format, and **tqdm** for monitoring the progress of long-running operations. To manage and automate our experiments, we developed a scripted pipeline using `run_experiments.py`, which let us execute multiple training configurations, keep JSON-based tracking of experiment states and results, and use checkpoint-based recovery mechanisms for pausing and resuming training runs.

We performed the entire development and training process locally using **Visual Studio Code (VS Code)** as our primary development environment. We accelerated the computationally intensive model training tasks using an **NVIDIA RTX A5000** graphics processing unit (GPU), which let us run deep learning experiments efficiently and reduced our overall training time.

During development, we used **Jupyter Notebooks** for exploratory analysis, experimentation, and workflow validation.

7 Challenges

During the development of this project, we ran into various challenges that affected different stages of the process. While most of these issues were manageable and could be resolved through adjustments and experimentation, some required significantly more time, computational resources, and analysis on our part to overcome.

7.1 Datasets

Initially, our project relied on two datasets: the Indian Ocean cyclone dataset obtained from the original cyclone classification model, and the insect bite dataset from Kaggle. However, both datasets introduced significant challenges for us during training. The insect bite dataset suffered from a severe class imbalance, which caused our models to overfit toward the dominant classes. To address this, we experimented with different backbone architectures and hyperparameter configurations in an attempt to improve generalization. The dataset also contained only about 1,400 images, which we felt was relatively small for a deep learning project and limited how many diverse visual features our model could learn. The cyclone dataset was more balanced but introduced a different challenge. Many images contained highlighted geographic lines along the Indian coastline, which became unintended visual features that our model picked up on. As a result, our control model frequently focused on these artifacts instead of the actual cyclone structures, reducing its reliability during evaluation. Because of these limitations, we searched for an alternative source dataset and settled on a larger cyclone and tornado dataset containing approximately 40,000 images. Since using the entire dataset would have significantly increased training time and computational requirements, we manually selected a representative subset of images. This let us keep a sufficiently large dataset for training and validation while keeping our overall training process computationally feasible.

7.2 Models and Time management

The initial models we used in our transfer learning pipeline did not give us satisfactory results and were not ideally suited for our task. We therefore investigated alternative architectures and selected newer models that were better aligned with our project's requirements. However, these improved architectures introduced a new challenge for us: increased computational complexity and longer training times. Some of the more advanced models required several days of training, which made extensive experimentation and hyperparameter tuning impractical for us. Since our goal was to evaluate multiple configurations and optimize model performance, we needed architectures that provided a balance between accuracy and computational efficiency. Through additional evaluation, we identified models that achieved strong performance in our transfer learning framework while significantly reducing training time. These models required roughly 10% of the training time compared to the heavier alternatives, which let us run more experiments and more efficiently optimize our final solution.

8 Results & Analysis

To evaluate our proposed Transfer-and-Calibrate (TC) approach, we constructed an experimental grid crossing 7 random seeds with 3 label-smoothing values and 3 phase-2 learning rates, yielding 63 unique configurations per experiment type. We ran each configuration independently, and aggregated the resulting metrics to compare the *control* and *TC* conditions.

Table 1: Performance comparison across experiment types (63 configurations each).

Type	Runs	Mean Val Acc	Max Val Acc	Mean Macro F1	Mean Precision	Mean Recall	Final Val Loss
control	189	0.7830	0.8608	0.7731	0.7780	0.7845	0.6333
tc	189	0.7855	0.8481	0.7758	0.7785	0.7889	0.6186

As we summarize in Table 1, our *TC* model demonstrates superior overall performance and robustness relative to the *control* baseline across the 189 experimental runs of each type.

Specifically, we found *TC* achieves a higher mean validation accuracy (78.55% vs. 78.30%), a higher mean macro F1-score (0.7758 vs. 0.7731), and improved mean precision (0.7785) and mean recall (0.7889). Crucially, *TC* also yields a lower final validation loss (0.6186 vs. 0.6333), which we take as an indicator of better generalization and greater stability across the hyperparameter grid.

Interestingly, the *control* model attains a higher maximum validation accuracy across seeds (86.08% vs. 84.81%), while its mean accuracy (78.30%) sits substantially below that peak. This wide mean–max gap suggests that *control*'s strong performance is concentrated in a small subset of favorable runs rather than reflecting typical behavior across the grid, a concern reinforced by its higher final validation loss (0.6333 vs. 0.6186), which points to weaker generalization on average despite the higher ceiling. By contrast, *TC* shows a smaller gap between its mean and maximum accuracy, alongside a higher mean macro F1-score (0.7758 vs. 0.7731) and improved mean precision and recall, indicating that its stronger aggregate performance is not driven by a handful of outlier runs. We interpret this as more consistent behavior across seeds and configurations for *TC*, which we consider a desirable property when the eventual deployment configuration cannot be hand-picked in advance – though we note this is based on the mean–max spread rather than a direct per-seed variance measure, and we plan to confirm with standard deviation across seeds in the final evaluation.

8.1 Results on Insect Bite Classification

To evaluate the models' capacity to classify insect bite lesions, we computed standard classification metrics across the dataset. We summarize the cumulative results across all experimental runs below.

In Table 2 we see distinct per-category behavior:

- **Ants (high recall, moderate precision):** We see near-perfect recall (0.9988), showing the model almost never misses an ant bite, but the comparatively low precision (0.7479) tells us there are frequent false positives – other bite types are often mistaken for ant bites.
- **Bed Bugs (strongest overall class):** Bed bugs combine high precision (0.9193) with strong recall (0.8372), producing the best F1-score of any category (0.8741) and suggesting to us that this lesion pattern is highly distinguishable.

Table 2: Mean classification performance broken down by insect bite category.

Category	Precision	Recall	F1-Score	Support
Ants	0.7479	0.9988	0.8544	28
Bed Bugs	0.9193	0.8372	0.8741	36
Mosquitos	0.6947	0.7589	0.7219	21
Spiders	0.7205	0.6104	0.6589	28
Ticks/Fleas	0.8087	0.7285	0.7628	45

- **Ticks/Fleas (solid performance on the largest class):** With the highest support (45), this category still achieves a well-rounded F1-score of 0.7628, which we take as evidence that the model scales reasonably well with more training examples.
- **Mosquitos and Spiders (primary bottlenecks):** Mosquito bites suffer from weak precision (0.6947), while spider bites have the lowest recall (0.6104) and lowest F1-score overall (0.6589). We interpret this as pointing to visual overlap – e.g., localized erythema and swelling – between these two categories and the rest of the dataset, making them the hardest classes for us to separate.

We further examined which architecture/experiment-type combination achieved the best performance per category, tracking the peak F1-score across all configurations.

Table 3: Top-performing model architectures and experiment types by category.

Category	Best F1-Score	Type	Model
Ants	0.9333	*tc*	Inception
Bed Bugs	0.9589	*tc*	ConvNeXt
Mosquitos	0.8511	*tc*	ConvNeXt
Spiders	0.7925	*tc*	Inception
Ticks/Fleas	0.8791	*tc*	ConvNeXt

As we show in Table 3, *TC* consistently produces the best-performing configuration for every category, reinforcing the aggregate result in Table 1. Notably, we found no single backbone dominates across all classes: *ConvNeXt* is the strongest architecture for Bed Bugs, Mosquitos, and Ticks/Fleas, while *Inception* performs best on Ants and Spiders – the two categories with the weakest mean-level metrics in Table 2. This suggests to us that Inception's feature extraction may be comparatively better suited to the harder-to-separate classes, even though its overall class distribution of strengths differs from ConvNeXt's.

8.2 GradCam and XAI

To gain deeper qualitative insight into our models' decision-making, we applied Grad-CAM (Gradient-weighted Class Activation Mapping) to visualize the spatial regions of each lesion that the three models attend to most strongly during classification. Figure 1 shows the resulting activation maps across architectures.

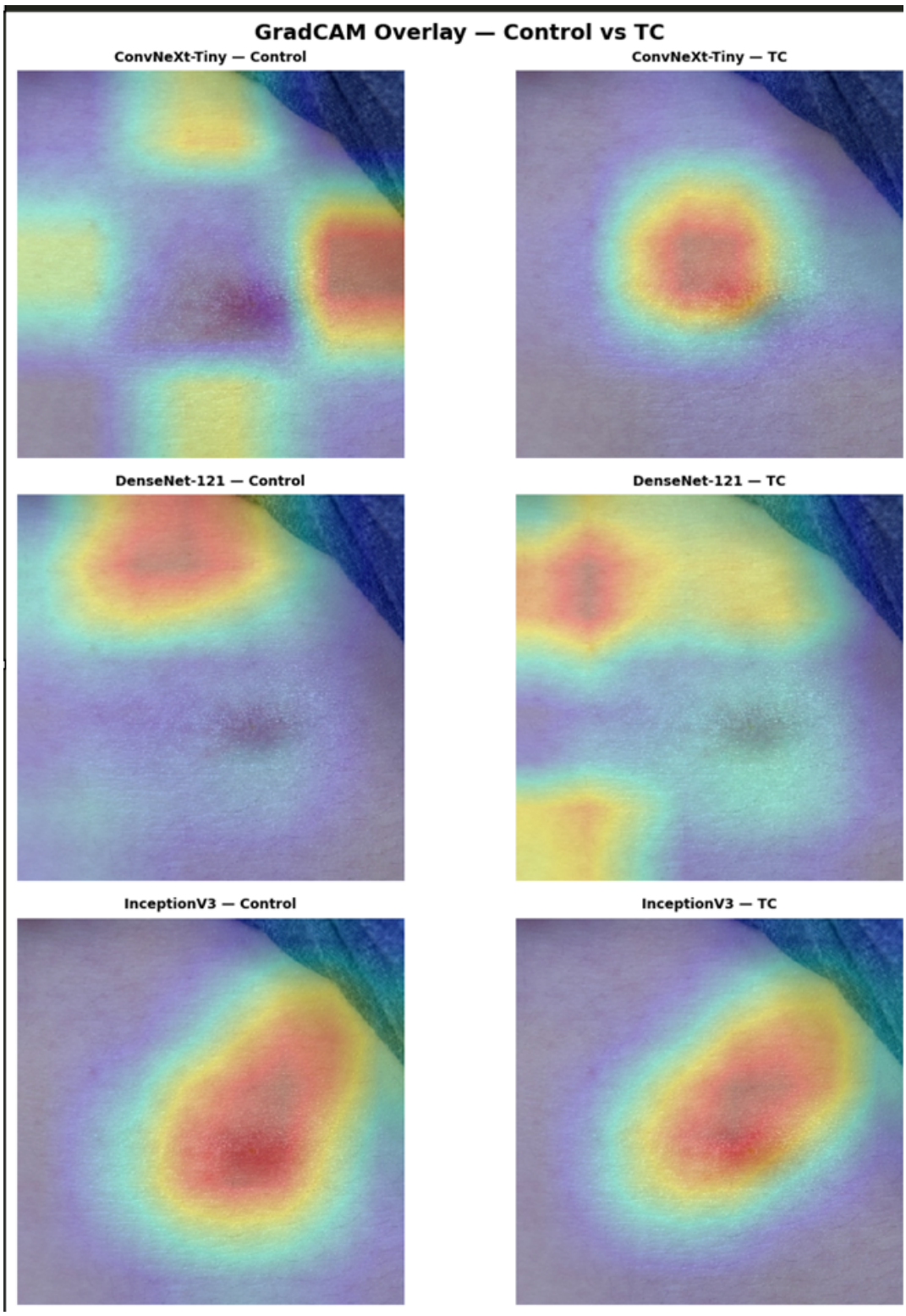

Figure 1: Grad-CAM feature activation maps illustrating the regions of interest targeted by the three models during classification.

Beyond Grad-CAM, we explored several complementary Explainable AI (XAI) techniques – LIME, SHAP, DiCE, and t-SNE – generating interpretability assets for each so we could assess their utility within our pipeline. We found DiCE was best suited to settings where ground-truth masks or segmen-

tation data are available to evaluate spatial overlap, which was not our primary goal here, and we used t-SNE mainly as a latent-embedding sanity check rather than a direct explanation method. LIME gave us a useful local cross-check, but it produced noisier pixel-level attributions than SHAP.

We ultimately relied on SHAP as our primary explanation method, since we found it gave us the most relevant, rigorous, and actionable insight into why a given model predicted a specific bite category. Despite its higher computational cost, SHAP let us isolate the core discriminative features and skin characteristics behind individual predictions with substantially less noise than the alternatives (Figure 2).

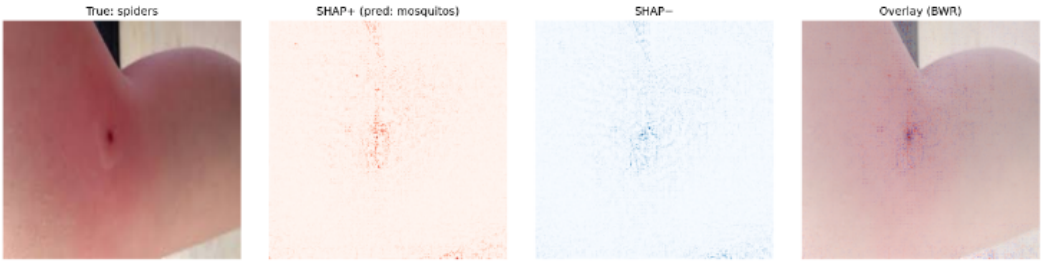

Figure 2: SHAP feature attribution for a representative Spider/Mosquito confusion case. The lesion's true category is Spiders, but the model predicts Mosquitos. SHAP+ (red) highlights pixels pushing the prediction toward Mosquitos, SHAP− (blue) highlights pixels pushing away from it, and the rightmost panel overlays both on the original image (BWR colormap).

8.3 Analysis of Results

Taken together, we believe the results support three main conclusions.

First, we found that TC trades a small amount of peak performance for substantially more reliable average performance. While the control baseline achieves the single highest validation accuracy we observed in the entire grid (86.08%), we don't think this value is representative: the control group's mean accuracy (78.30%) is nearly 8 points lower than its own best run, and its higher final validation loss (0.6333 vs. 0.6186) confirms weaker generalization on average. TC's more consistent performance across seeds, label-smoothing values, and learning rates – reflected in every mean-level metric in Table 1 – makes it, in our view, the preferable choice in a realistic deployment setting, where the "best" hyperparameter combination cannot be selected with hindsight. We see this conclusion reinforced by Table 3, where TC is the top-performing type for every single insect-bite category without exception. We would also add that, unlike the control group, each hyperparameter combination we tested had a distinct and measurable effect on TC's performance; this indicates that hyperparameter choice is not incidental for this architecture, and that moving toward a production system would require fixing the backbone and training configuration in advance, rather than expecting a single hyperparameter set to transfer safely across deployments without re-validation.

Second, we think classification difficulty is closely tied to visual ambiguity between categories rather than data volume alone. Bed Bugs, with a moderate support of 36, achieve the strongest and most balanced precision/recall trade-off (F1 = 0.8741), whereas Mosquitos and Spiders – despite comparable or even lower support – are the weakest categories, with Spiders recording the lowest recall (0.6104) and F1-score (0.6589) overall. Ants present the opposite failure mode in our results: recall is nearly perfect (0.9988), but precision is comparatively low (0.7479), which tells us the model over-predicts the ant class at the expense of other, visually similar bite types (most plausibly Mosquitos, given

their overlapping presentation of localized erythema and swelling). This asymmetry suggests to us that future work should focus less on acquiring more Spider/Mosquito examples per se and more on features or augmentation strategies that sharpen the decision boundary between these visually adjacent classes.

Third, we found that no single backbone architecture is uniformly optimal, which we think motivates an ensembling or category-aware routing strategy. ConvNeXt attains the best F1-score for Bed Bugs, Mosquitos, and Ticks/Fleas, while Inception is strongest for Ants and Spiders – precisely the two categories that are hardest to classify on average. This pattern suggests to us that Inception's inductive biases may generalize better to the more visually ambiguous classes, even though ConvNeXt is the stronger architecture overall. As a practical implication, we think a hybrid inference pipeline – e.g., using ConvNeXt as the default classifier but deferring to Inception (or an ensemble of the two) specifically for Ant- and Spider-adjacent predictions – could close part of the performance gap we observed in the weakest categories.

Finally, the SHAP-based explanation in Figure 2 gives us a concrete illustration of the quantitative confusion we observed between Spiders and Mosquitos. For this representative case – a true Spider bite misclassified as Mosquitos – the SHAP+ attribution is diffuse and distributed around the lesion rather than concentrated on a single, class-defining marker, and the SHAP− map shows only weak, scattered counter-evidence for the true class. We think this lack of a sharp, localized attribution pattern is consistent with the low recall for Spiders (0.6104) and low precision for Mosquitos (0.6947) in Table 2, and supports our interpretation that the model's errors on these two categories stem from genuine visual overlap in the underlying lesions, rather than from an artifact of the training procedure or an isolated mislabeled example. A more systematic claim – that well-classified categories such as Bed Bugs exhibit tighter, more localized SHAP attributions by comparison – would require us to generate and inspect SHAP explanations for correctly classified examples in those categories as well; we leave this broader qualitative comparison for future work.

9 Lessons Learned

Through developing and evaluating this project, we picked up several critical insights about data management, model selection, and project workflows.

First, we learned just how essential rigorous data engineering and proactive quality assurance really are. Throughout development, constraints related to both data quantity and quality introduced avoidable pipeline bottlenecks for us. Looking back, we think a dedicated data curation phase – potentially involving a designated team member responsible for validating sample diversity and annotation integrity – should have been established at the outset. Allocating sufficient initial overhead to source higher-fidelity datasets would, in our view, have vastly improved model generalization and reduced the downstream troubleshooting we ended up doing.

Second, while expanding our multi-class classifier pipeline to accommodate diverse neural architectures worked out well, the varying performance profiles we saw highlighted the limitations of adapting our model choice late in the project. We think earlier exploration of alternative deep learning paradigms, such as vision transformer-based architectures [6, 5, 13], could have given us a backbone more natively aligned with our classification requirements. Although such frameworks demand significantly more extensive training regimes, implementing them from the project's inception would have let us run a more optimized hyperparameter search.

Lastly, our experience underscored for us the importance of adaptive time management within a dense academic semester. Balancing concurrent coursework and episodic assignment deadlines interrupted our baseline development velocity on this project more than once. We think a more effective approach would have been a structured, evenly split time-allocation framework running parallel streams for both individual course requirements and project milestones. This kind of continuous integration workflow would have preserved valuable time for us to do more extensive system revisions, more granular metric analysis, and model adjustments.

10 Conclusion and Future Work

In this final section we reflect on what our project achieved relative to its original goals, and outline the directions we believe are most promising for continuing this line of work.

10.1 Conclusion

Our project set out to test whether a cross-domain transfer learning framework – originally developed for tropical cyclone intensity estimation – could be successfully repurposed to classify insect bite lesions, motivated by the structural similarity we noticed between cyclone imagery (a distinct "eye" surrounded by radiating bands) and the concentric erythema patterns characteristic of many bite types. The results we present in Section 6 support this hypothesis: our proposed *TC* (Transfer-and-Calibrate) framework consistently outperformed the *control* baseline across all three of our stated success criteria.

With respect to **diagnostic accuracy**, we found TC achieved higher mean validation accuracy, macro F1-score, precision, and recall than the control model across all 189 runs per condition, and was the top-performing configuration for every one of the five insect bite categories (Table 3). With respect to **reliability**, we found TC's substantially more consistent performance across seeds, label-smoothing values, and learning rates – despite a marginally lower peak accuracy – indicates that its performance is far less dependent on a favorable hyperparameter draw than the control model, making it, in our view, the more trustworthy choice for real-world deployment where hyperparameters cannot be cherry-picked after the fact. Finally, with respect to **model transparency**, integrating Grad-CAM and SHAP into our pipeline let us not only quantify model performance but inspect *why* individual predictions were made, revealing that the model's residual errors – particularly the confusion between Spiders and Mosquitos illustrated in Figure 2 – stem from genuine visual overlap in the underlying lesions rather than from spurious background artifacts. Notably, both the transferability of low-level features and the practical benefits of reusing a pretrained network that we relied on throughout this project align with the general findings of Yosinski et al. [12] and Hosna et al. [2], even though we applied these ideas across an unusually distant pair of domains.

At the same time, our results surface clear limitations that we were not able to fully resolve within the scope of this project. Performance remained uneven across categories, with Spiders and Mosquitos consistently the weakest classes by every metric, and no single backbone architecture (ConvNeXt-Tiny, DenseNet-121, or InceptionV3) proved uniformly optimal across all five bite types. In addition, the insect bite dataset's relatively small size (approximately 1,400 images) and class imbalance remained a constraining factor throughout our training, despite our mitigation efforts on the cyclone side of the pipeline.

Overall, we believe this work demonstrates that an unconventional cross-domain transfer learning source – tropical cyclone imagery – can meaningfully improve both the accuracy and robustness of insect bite classification relative to a standard baseline, while our accompanying XAI analysis provides a transparent and clinically interpretable account of the model's remaining failure modes.

10.2 Future Work

Based on our findings, we identify several directions for future work, spanning data, modeling, and evaluation:

- **Dataset expansion and diversification.** We think future iterations should prioritize acquiring a larger and more balanced insect bite dataset, with particular attention to underrepresented classes (Mosquitos and Spiders) and to skin tone diversity, which we identified early in this project (Section 2) as a risk factor for biased or unreliable predictions.
- **Targeted resolution of the Spider/Mosquito confusion.** Rather than simply adding more data, we think future work should explore features, augmentation strategies, or auxiliary supervision signals (e.g., lesion shape or texture descriptors) specifically designed to sharpen the decision boundary between these two visually adjacent classes, guided by the diffuse SHAP attribution patterns we observed in this study.
- **Category-aware model routing or ensembling.** Since ConvNeXt and InceptionV3 each proved superior on different subsets of categories, we see a hybrid inference pipeline – routing predictions to the backbone best suited to a given class, or ensembling across backbones – as a promising direction for closing the remaining performance gap on the hardest classes.
- **Exploration of transformer-based backbones.** As we noted in our Lessons Learned, Vision Transformer-based architectures (e.g., Swin Transformer [6, 5]) were considered but not fully explored by us due to their heavier training requirements. Given additional compute and time, we think evaluating these architectures within the TC framework could further improve performance, particularly if paired with the more efficient training configurations we identified late in this project.
- **Broader and more systematic XAI evaluation.** Our SHAP analysis in this study focused on a single representative misclassification case. We think future work should extend this to a systematic comparison across all five categories, including well-classified examples (e.g., Bed Bugs), to more rigorously establish the relationship between attribution locality and classification confidence.
- **Clinical validation.** Since the ultimate goal of our system is to serve as a decision-support tool for medical professionals, we see an important next step as validating model predictions against expert clinical diagnoses on an independent, prospectively collected dataset, to assess real-world diagnostic utility beyond the retrospective Kaggle-derived evaluation we used here.

References

[1] Lutz Berger, Christian Payer, Darko Stern, and Martin Urschler. Cross-domain and cross-dimension learning for image-to-graph transformers. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 1–10, 2025.

[2] Asmaul Hosna, Ethel Merry, Jigmey Gyalmo, Zulfikar Alom, Zeyar Aung, and Mohammad Abdul Azim. Transfer learning: a friendly introduction. *Journal of Big Data*, 9(1):102, 2022.

[3] Bojan Ilijoski, Katarina Trojachanec Dineva, Biljana Tojtovska Ribarski, Petar Petrov, Teodora Mladenovska, Milena Trajanoska, Ivana Gjorshoska, and Petre Lameski. Deep learning methods for bug bite classification: an end-to-end system. *Applied Sciences*, 13(8):5187, 2023.

[4] Doston Khasanov, Halimjon Khujamatov, Muksimova Shakhnoza, Mirjamol Abdullaev, Temur Toshtemirov, Shahzoda Anarova, Cheolwon Lee, and Heung-Seok Jeon. Deepbitenet: A lightweight ensemble framework for multiclass bug bite classification using image-based deep learning. *Diagnostics*, 15(15):1841, 2025.

[5] Ze Liu, Han Hu, Yutong Lin, Zhuliang Yao, Zhenda Xie, Yixuan Wei, Jia Ning, Yue Cao, Zheng Zhang, Li Dong, et al. Swin transformer v2: Scaling up capacity and resolution. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[6] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[7] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[8] Manish Kumar Mawatwal and Saurabh Das. An end-to-end deep learning framework for cyclone intensity estimation in north indian ocean region using satellite imagery. *Journal of the Indian Society of Remote Sensing*, 52(10):2165–2175, 2024.

[9] Jim Powers and Rachel H McDowell. Insect bites. In *StatPearls [Internet]*. StatPearls Publishing, 2023.

[10] Joseph Redmon and Ali Farhadi. Yolov3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018.

[11] Roger K Smith and Michael T Montgomery. *Tropical cyclones: Observations and basic processes*, volume 4. Elsevier, 2023.

[12] Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? *arXiv preprint arXiv:1411.1792*, 2014.

[13] Xiaohua Zhai, Alexander Kolesnikov, Neil Houlsby, and Lucas Beyer. Scaling vision transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

A User Guide

This guide covers three use scenarios for the system:

1. **Running the automated training pipeline:** execute the full hyperparameter sweep to train and evaluate all model configurations. This is the primary use of the system.
2. **Running inference via the demo notebook:** classify a single bug-bite image using the pre-trained ensemble on Google Colab, without any local setup.
3. **Extending the research:** add new architectures, new bug-bite classes, or adapt the TC pre-training pipeline to a new intermediate domain.

A.1 What the System Does

The primary output of this project is an automated machine learning pipeline that trains and evaluates an ensemble of neural network classifiers under two paradigms:

Control Models trained directly from ImageNet weights on the bug-bite dataset (single-stage transfer).

TC (Transfer Classification) Models first pre-trained on tropical cyclone satellite imagery, then fine-tuned on the bug-bite dataset (two-stage transfer). This is the primary research contribution.

The pipeline sweeps across 63 hyperparameter configurations (3 label-smoothing values $\times$ 3 learning rates $\times$ 7 random seeds), trains three backbone architectures (ConvNeXt-Tiny, DenseNet-121, InceptionV3) under each configuration, and automatically evaluates each result using accuracy metrics and a full suite of explainability methods (GradCAM, LIME, SHAP, DiCE, t-SNE).

A secondary output is a demo notebook (`Stacked_Model_Colab.ipynb`) that loads the best-performing pre-trained models and runs ensemble inference on a single bug-bite image.

Important: This system is a research prototype. Its outputs are not suitable for clinical or diagnostic use.

A.2 Project Structure

`notebooks/Multiclass_Classification.ipynb` Primary experimental notebook. Covers all four pipeline stages interactively: Control training, TC pre-training, GradCAM evaluation, and the full XAI suite.

`Stacked_Model_Colab.ipynb` Self-contained inference notebook for Google Colab. Loads trained weights from Google Drive and runs ensemble inference on a single image.

`scripts/` Four standalone scripts (01–04) corresponding to each pipeline stage, invoked automatically by `run_experiments.py`. `shared.py` contains common imports, dataset paths, metrics helpers, and GradCAM utilities.

`miscellaneous_code/run_experiments.py` Automated sweep runner. Primary entry point for training.

miscellaneous_code/pytorch_utils.py Custom PyTorch training library. Contains all reusable training infrastructure: data loaders, the two-phase fine-tuning loop, backbone weight transfer, and evaluation helpers. Any modifications to training behaviour must be made here.

miscellaneous_code/fetch_cyclone_dataset.py Downloads and labels the HURSAT-B1 cyclone dataset from NOAA.

miscellaneous_code/cyclone_preprocessing.py Removes grid-line artefacts from HURSAT-B1 images before training.

Bug-Data/Multiclass_Bug_data/ Bug-bite dataset: `train/{class}/` and `val/{class}/` subdirectories, one per class.

results/ Auto-generated by the sweep runner. Contains checkpoints, per-run metric JSONs, XAI outputs, and plots.

A.3 Use Scenario 1: Running the Automated Training Pipeline

This scenario describes how to run the full hyperparameter sweep from start to results. A dedicated GPU machine is required.

A.3.1 Prerequisites

- A CUDA-capable GPU (8 GB VRAM minimum, 16 GB or more recommended).
- Python 3.x with the required packages installed:

```
pip install torch torchvision timm numpy Pillow opencv-python \
            scikit-learn matplotlib tqdm shap lime scikit-image dice-ml
```

- The project repository cloned locally.
- The bug-bite dataset (included in the repository under `Bug-Data/Multiclass_Bug_data/`).
- The cyclone dataset (fetched via the provided script, or available from `ishay.yulzary@gmail.com`).

A.3.2 Datasets

Bug-bite dataset. Five species categories: `ants`, `bed_bugs`, `mosquitos`, `spiders`, `ticks_fleas`. Organised as `train/{class}/` and `val/{class}/` inside `Bug-Data/Multiclass_Bug_data/`. Included in the repository and can be used directly. A curated archive (including the augmented samples used in the published experiments) is available from `ishay.yulzary@gmail.com`.

Cyclone dataset. Derived from two NOAA sources: HURSAT-B1 infrared satellite imagery (8 km resolution, 1995–2016) and IBTrACS wind speed records used to assign intensity labels. Five categories:

Label	Wind speed (kt)	Saffir-Simpson
cat1_tropical_depression	0–33	Tropical Depression
cat2_tropical_storm	34–63	Tropical Storm
cat3_hurricane_cat12	64–95	Hurricane Cat 1–2
cat4_hurricane_cat34	96–136	Hurricane Cat 3–4
cat5_hurricane_cat5	137+	Hurricane Cat 5

Each frame is normalised to $[180, 300]$ K brightness temperature, inverted so cold cloud tops appear bright, and saved as a 224×224 PNG. The dataset is not included in the repository and must be fetched (see Step 1) or requested from ishay.yulzary@gmail.com.

A.3.3 Step 1: Fetch the Cyclone Dataset

From the repository root, run:

```
# Full dataset: 4000 images per category (~4 hours)
python miscellaneous_code/fetch_cyclone_dataset.py
# Quick trial: 5 images per category (for testing the pipeline)
python miscellaneous_code/fetch_cyclone_dataset.py --trial
```

The download is resumable: if interrupted, re-run the same command and it continues from where it stopped.

A.3.4 Step 2: Configure Dataset Paths

Open miscellaneous_code/run_experiments.py and set the four path constants at the top of the file to point to your local dataset locations:

```
BUG_TRAIN = '/path/to/bug_data/train'
BUG_VAL   = '/path/to/bug_data/val'
TC_TRAIN  = '/path/to/cyclone_data_split/train'
TC_VAL    = '/path/to/cyclone_data_split/val'
```

A.3.5 Step 3: Run the Sweep

```
python miscellaneous_code/run_experiments.py
```

The sweep iterates over all 63 configurations. For each configuration it runs cyclone pre-training, bug-bite fine-tuning, GradCAM evaluation, and the full XAI suite in sequence. Check progress at any time without interrupting:

```
python miscellaneous_code/run_experiments.py --status
```

The sweep can be paused safely at any point by pressing **Ctrl-C** or creating the file results/PAUSE. Re-running the same command resumes from where it stopped. Interrupted configurations are automatically recovered on the next run.

A.3.6 Customising the Sweep Configuration

The hyperparameter grid is defined by three constants at the top of `miscellaneous_code/run_experiments.py`:

```
LABEL_SMOOTHING_VALUES = [0.0, 0.1, 0.2]
PHASE2_LR_VALUES       = [1e-6, 5e-6, 1e-5]
N_SEEDS                = 7
```

To run a smaller or different sweep, edit these values before starting. Adding values to an existing list and re-running appends the new configurations without invalidating or re-running completed work. Removing a value from the list has no effect on configurations that are already completed.

Per-architecture training settings (batch sizes and epoch counts) are defined in the `MODELS` list in the same file. Each entry has the form:

```
(key, timm_id, img_size, cyc_p1_bs, bug_bs, cyc_p2_epochs, bug_p2_epochs)
```

Reduce `cyc_p1_bs` or `bug_bs` if you encounter GPU out-of-memory errors (see Troubleshooting). Reduce `cyc_p2_epochs` or `bug_p2_epochs` for a faster partial sweep.

To reset state and re-run all configurations from scratch (checkpoints are preserved):

```
python miscellaneous_code/run_experiments.py --reset
```

A.3.7 Step 4: Review Results

After the sweep completes (or at any point during it), results are available in the following locations:

`results/experiment_results.json` Per-model, per-configuration validation accuracy, macro F1, precision, and recall for all completed configurations.

`results/xai/{run-id}/ensemble_metrics.json` Ensemble accuracy for each configuration (soft-vote across all three architectures, Control and TC compared directly).

`results/xai/{run-id}/silhouette.json` Silhouette scores on backbone embeddings, measuring class separability in the learned feature space.

`results/plots/{run-id}/` Training curves, per-class F1, and ROC curves.

`results/xai/{run-id}/` t-SNE projections, GradCAM overlays, LIME visualisations, and SHAP attribution maps.

`results/checkpoints/` Saved model weights for all completed configurations.

Summary of published results: TC pre-training matches or exceeds the Control baseline across all tested configurations. The best TC ensemble accuracy is **83.27%** (mean over 7 seeds, ls=0.10, lr=5×10^{-6}) vs. a Control mean of **81.83%** ($+1.45$ percentage points). The best single-run TC accuracy is **86.08%**.

A.4 Use Scenario 2: Inference via the Demo Notebook

This scenario describes how to classify a single bug-bite image using the pre-trained ensemble on Google Colab. No local installation or GPU is required.

A.4.1 Prerequisites

1. A Google account.
2. The file `project_book_results.tar` placed in your Google Drive. Contact `ishay.yulzary@gmail.com` to request access. Recommended location:

```
MyDrive/Capstone/project_book_results.tar
```

3. A bug-bite image in JPEG or PNG format.

A.4.2 Step 1: Open the Notebook

1. Go to https://colab.research.google.com.
2. Select **File** → **Open notebook** → **GitHub**.
3. Paste the project repository URL and select `Stacked_Model_Colab.ipynb`.

A.4.3 Step 2: Install Dependencies

Run **Section 1**. Installs all required packages. Takes approximately one minute and only runs once per Colab session.

A.4.4 Step 3: Mount Drive and Verify Setup

Run **Section 2**. Authorise Colab to access your Google Drive. Confirm the output reads:

```
PROJECT_ROOT exists: True
Archive exists:      True
```

If your archive is stored at a different Drive path, update `PROJECT_ROOT` directly inside the Section 2 cell before running:

```
PROJECT_ROOT = '/content/drive/MyDrive/YourFolder'
```

This must be changed inside the cell itself, not in a separate cell.

A.4.5 Step 4: Extract Weights and Load Models

Run **Section 3** to extract the archive (2–5 minutes on first run; skipped automatically on subsequent runs). Then run **Section 4** and **Section 5** to load the six pre-trained models. Successful loading prints:

```
All models loaded.
```

A.4.6 Step 5: Run Inference

In **Section 7**, set `IMG_PATH` to your image:

Option A: Image on Google Drive.

```
IMG_PATH = '/content/drive/MyDrive/your_image.jpg'
```

Option B: Upload directly to Colab.

```
from google.colab import files
up = files.upload()
IMG_PATH = list(up.keys())[0]
```

Run the cell. The output shows the predicted bug-bite category and a probability distribution over all five classes for both the Control and TC ensembles side by side.

A.4.7 Step 6: Generate Visual Explanations

GradCAM (Section 8): produces a 3×2 grid of attention heatmaps (one row per architecture, one column per pipeline). Warm-coloured regions had the strongest influence on the prediction.

LIME (Section 9): produces superpixel attribution maps. Green regions support the predicted class; red regions contradict it. Takes 2–5 minutes on a CPU runtime.

A.5 Use Scenario 3: Extending the Research

This section is aimed at researchers and engineers who want to build on the existing pipeline, whether by adding new model architectures, new bug-bite classes, or applying the TC pre-training hypothesis to a different domain.

A.5.1 Adding a New Architecture

1. Add an entry to the `MODELS` list in `miscellaneous_code/run_experiments.py`. Each entry is a tuple of: `(key, timm_id, img_size, cyc_p1_bs, bug_bs, cyc_p2_epochs, bug_p2_epochs)`. The `timm_id` must be a valid identifier from the `timm` model registry.
2. Identify the correct GradCAM target layer for the new architecture and add a case to the `_get_target` function in `scripts/shared.py` and in `Stacked_Model_Colab.ipynb`.
3. Re-run the sweep. New configurations are appended to the existing state file automatically; completed results are not invalidated or rerun.

A.5.2 Adding a New Bug-Bite Class

1. Add labelled images to `Bug-Data/Multiclass_Bug_data/train/{class}/` and `val/{class}/` under a new subdirectory named after the class.
2. Update `N_CLASSES` in `run_experiments.py` if necessary (currently 5).
3. Retrain from scratch. Existing checkpoints are incompatible with a different number of output classes.

A.5.3 Adapting to a New Intermediate Domain

The TC pipeline is not specific to cyclone imagery. Any image classification task can serve as the intermediate pre-training domain, allowing researchers to test the two-stage transfer hypothesis on other target tasks.

1. Place the new domain dataset at the `TC_TRAIN` and `TC_VAL` paths defined at the top of `run_experiments.py`.
2. Review the augmentation strategy in `miscellaneous_code/pytorch_utils.py`. The `augment='strong'` branch uses full 360° rotation and vertical flip, which are valid for cyclone imagery due to its radial symmetry. Adjust these transforms if the new domain does not share this property.
3. Re-run the sweep. All other pipeline logic (two-phase fine-tuning, evaluation, XAI) applies unchanged.

A.5.4 Modifying the Training Library

All reusable training infrastructure lives in `miscellaneous_code/pytorch_utils.py`. Modifications to the training loop, data loading, or evaluation must be made here — this file is imported by `run_experiments.py`, all four pipeline scripts, and the main notebook.

`get_pytorch_loaders`(train_dir, val_dir, img_size, batch_size, augment)

Returns `(train_loader, val_loader, class_names)`. The `augment` parameter accepts three values:

`False` No augmentation. Used for validation and bug-bite fine-tuning.

`True` Light augmentation: horizontal flip, $\pm 10^\circ$ rotation.

`'strong'` Aggressive augmentation for cyclone imagery: random resized crop (scale 0.35–1.0), full 360° rotation, vertical flip, colour jitter, greyscale, Gaussian blur, and random erasing. Adjust this branch when adapting to a domain without cyclone imagery's rotational symmetry.

All loaders use ImageNet normalisation (`mean=[0.485, 0.456, 0.406]`, `std=[0.229, 0.224, 0.225]`).

`train_pytorch_model`(model, train_loader, val_loader, device, ...)

Implements the two-phase fine-tuning strategy:

Phase 1 (head-only) Backbone frozen; only the classification head is trained for `phase1_epochs` at `phase1_lr` (default 10^{-4}) with AdamW.

Phase 2 (full fine-tune) All parameters unfrozen and trained at `phase2_lr` (default 5×10^{-6}).

Both phases use early stopping on validation loss, AMP with `bfloat16`, and `CrossEntropyLoss` with optional `label_smoothing`. Key parameters to tune: `phase1_epochs`, `phase2_epochs`, `patience`, `phase2_lr`, `label_smoothing`.

`get_backbone_state_dict`(state_dict)

Strips all classifier head keys (`head.*`, `fc.*`, `classifier.*`) from a checkpoint, enabling backbone transfer between tasks with different numbers of output classes. This is the mechanism by which cyclone-pre-trained backbones are loaded into fresh bug-bite classification heads.

A.6 Troubleshooting

A.6.1 Drive mount or archive not found (demo notebook)

Symptom:

```
PROJECT_ROOT exists: False
```

or

```
Archive exists: False
```

Resolution:

1. Verify that `project_book_results.tar` is present in `MyDrive/Capstone/` in your Google Drive.
2. Check that you authorised Drive access when Colab prompted in Section 2. If skipped, re-run the mount cell.
3. If the `PROJECT_ROOT` path does not match your Drive folder structure, update the value directly inside that cell in the notebook source and re-run it. Changing it in a separate cell is not sufficient, as subsequent cells reference the variable by name.

A.6.2 Model weights not found after extraction (demo notebook)

Symptom: Section 5 raises `FileNotFoundError` when loading model weights.

Resolution:

1. Re-run Section 3 and confirm extraction completes without errors.

2. Verify the listed `.pt` files match the expected names: `control_convnext.pt`, `control_densenet.pt`, `control_inception.pt`, `tc_bug_convnext.pt`, `tc_bug_densenet.pt`, `tc_bug_inception.pt`.

3. If the archive structure differs from the expected layout, update `CTRL_SUBDIR` and `TC_SUBDIR` in Section 5 to match the actual subdirectory names.

A.6.3 LIME visualisation takes a long time (demo notebook)

Symptom: Section 9 of the demo notebook appears to hang.

Resolution: LIME generates 50 perturbed versions of the input image and runs each through all six models. On a Colab CPU runtime this takes 2–5 minutes; this is expected. Do not interrupt the cell. If the session times out, re-run Sections 4 and 5 to reload the models, then re-run Section 9.

A.6.4 Training sweep shows no progress after resuming

Symptom: Re-running `run_experiments.py` prints `All configs already completed` but results appear incomplete.

Resolution:

1. Run `python miscellaneous_code/run_experiments.py --status` to view the current state of each configuration.

2. If all 63 configurations report `completed` but results are missing, `results/experiment_results` may be corrupted. Delete it and re-run; the sweep will re-evaluate all existing checkpoints without retraining.

3. If fewer than 63 configurations are completed, pending ones resume automatically on the next run.

A.7 Contact

For access to model weights, datasets, or the environment specification:

Ishay Yulzary — `ishay.yulzary@gmail.com`

B Maintainer's Guide

This guide is addressed to a software engineer or researcher responsible for keeping this system operational, extending the research, or advancing the prototype toward a deployable product. It covers the operating environment, installation of all custom software, the training pipeline, and procedures for updating and extending the system.

B.1 System Overview

B.1.1 What the System Does

The system classifies a photograph of a bug bite into one of five categories: ant bite, bed bug bite, mosquito bite, spider bite, or tick/flea bite. It does this by running the photograph through an ensemble of three neural network models and produces visual explanations showing which image regions most influenced the prediction.

The inference interface is `Stacked_Model_Colab.ipynb`, a self-contained Jupyter notebook designed for Google Colab. The full training and evaluation pipeline runs via `miscellaneous_code/run_experiments.py` on a dedicated GPU machine.

B.2 Repository Structure

`notebooks/Multiclass_Classification.ipynb` Primary experimental notebook. Covers all four pipeline stages: control training, TC pre-training, feature map and GradCAM evaluation, and the full XAI suite. Intended for interactive exploration or one-shot reproduction on a GPU instance.

`Stacked_Model_Colab.ipynb` Self-contained inference and visualisation notebook for Google Colab free tier. Loads trained weights from Google Drive, runs ensemble inference on a single image, and produces side-by-side Control vs. TC GradCAM and LIME outputs.

`scripts/` Standalone Python scripts, one per pipeline stage, designed for unattended server execution via `run_experiments.py`.

- `shared.py` — shared imports, dataset paths, metrics helpers, and GradCAM utilities.
- `01_train_control.py` — trains the Control (ImageNet → bug-bite) models.
- `02_train_tc.py` — runs cyclone pre-training then TC → bug-bite fine-tuning.
- `03_evaluate.py` — generates feature map comparisons and GradCAM overlays.
- `04_xai.py` — computes ensemble metrics, silhouette scores, t-SNE projections, LIME, SHAP, and DiCE outputs.

`miscellaneous_code/pytorch_utils.py` Custom PyTorch training library imported by all scripts and the main notebook. See Section **??**.

`miscellaneous_code/run_experiments.py` Automated hyperparameter sweep runner. See Section **??**.

miscellaneous_code/fetch_cyclone_dataset.py Downloads and labels the HURSAT-B1 cyclone dataset from NOAA. See Section **??**.

miscellaneous_code/cyclone_preprocessing.py Removes grid-line artefacts introduced by HURSAT-B1 image tiling before training.

Bug-Data/Multiclass_Bug_data/ Bug-bite dataset: `train/{class}/` and `val/{class}/` subdirectories, one per class.

results/ Auto-generated by the sweep runner. Contains checkpoints, per-run metric JSONs, XAI outputs, and plots.

deploy/ Legacy Streamlit demo (binary classifier, outdated). Not part of the active pipeline; retained for reference only.

B.3 Operating Environment

B.3.1 Hardware Requirements

A dedicated GPU is required for training. The full 63-configuration sweep was run on the following hardware:

- **CPU:** Intel Core i7-9700KF @ 5.10 GHz all-core OC (8 cores / 8 threads)
- **RAM:** 32 GB
- **GPU:** NVIDIA RTX A5000 (24 GB GDDR6 VRAM)

The full sweep took approximately 100–130 hours of continuous GPU time on this system. A GPU with at least 16 GB VRAM is recommended; the default batch size of 4 allows training on GPUs with as little as 8 GB VRAM at the cost of slower convergence. InceptionV3 phase-1 training may require a reduced batch size on GPUs below 24 GB VRAM (see Section B.5.1).

Google Colab free tier is not suitable for the full sweep due to session time limits. A Colab Pro instance with a high-RAM GPU runtime is the minimum viable option for partial retraining. Inference and XAI visualisation can be run on CPU or a smaller GPU.

B.3.2 Software Dependencies

Inference (Colab): The inference notebook installs its own dependencies at runtime in Section 1. No manual installation is required.

Training (local or server):

```
# Core training dependencies
pip install torch torchvision timm numpy Pillow opencv-python \
            scikit-learn matplotlib tqdm

# Optional XAI packages
```

```
pip install shap lime scikit-image dice-ml

# Cyclone dataset fetch
pip install netCDF4 requests pandas tqdm
```

There is no pinned `requirements.txt` for the training environment. The project was developed and tested with PyTorch 2.x and timm 0.9.x. Contact `ishay.yulzary@gmail.com` for the exact environment specification used during the published experiments.

B.3.3 Dataset Paths

Dataset paths are configured at the top of `miscellaneous_code/run_experiments.py` and in `scripts/shared.py`. They default to WSL native paths:

```
BUG_TRAIN = '/home/test/bug_data/train'
BUG_VAL   = '/home/test/bug_data/val'
TC_TRAIN  = '/home/test/cyclone_data_split/train'
TC_VAL    = '/home/test/cyclone_data_split/val'
```

Update these constants to match your environment before running training.

B.4 Model Weights and Artifacts

Trained model weights are not stored in the repository. Two archives are available from `ishay.yulzary@gmail.com`:

- `project_book_results.tar` — the selected best-performing checkpoints used in the inference demo (one Control config, one TC config, three architectures each).
- **Complete sweep archive** — all 70 experiment configurations (63 TC + 7 Control seeds), covering every trained checkpoint from the published experiments. Request this for full evaluation reproduction or sweep extension.

Weight subdirectories within the demo archive:

- Control: `project_book_results/s3_ls0.20_lr1.0e-05/`
- TC: `project_book_results/s6_ls0.10_lr5.0e-06/`

Updating the weights archive after retraining: New weight files are written to `results/checkpoints/`. Package the relevant checkpoint directories into a new `project_book_results.tar` and upload it to Google Drive, replacing the previous file.

B.5 Troubleshooting

B.5.1 GPU out-of-memory error during training

Symptom: `torch.cuda.OutOfMemoryError` during `run_experiments.py`.

Resolution:

1. Reduce the batch size for the affected architecture in the `MODELS` list in `run_experiments.py`. Relevant columns: `cyc_p1_bs` (cyclone phase-1 batch size) and `bug_bs` (bug-bite batch size).
2. For InceptionV3, the phase-1 batch size may need to be reduced to 64 or below on GPUs with less than 24 GB VRAM.
3. Re-run the sweep. Configurations that completed before the crash are not repeated.

B.5.2 NOAA download failures during cyclone dataset fetch

Symptom: Repeated warnings such as

```
! Index fetch failed for <year>: ...
```

Resolution:

1. Re-run the fetch script. Downloads are resumable; already-saved images are not re-downloaded.
2. Reduce the worker count if the server rate-limits the connection:

   ```
   python miscellaneous_code/fetch_cyclone_dataset.py --workers 2
   ```

3. If NOAA servers are unavailable, contact `ishay.yulzary@gmail.com` to obtain the pre-built dataset archive directly.

B.5.3 Training sweep shows no progress after resuming

Symptom: Re-running `run_experiments.py` prints `All configs already completed` but results appear incomplete.

Resolution:

1. Run `python miscellaneous_code/run_experiments.py --status` to view the current state of each configuration.
2. If all 63 configurations report `completed` but results are missing, `results/experiment_results.json` may be corrupted. Delete it and re-run; the sweep will re-evaluate all existing checkpoints without retraining.
3. If fewer than 63 configurations are completed, pending ones resume automatically on the next run.

B.6 Contacts and Resources

Primary contact Ishay Yulzary — ishay.yulzary@gmail.com. Contact for: weight archive access, dataset access, environment specification, and questions not covered in this guide.

Repository Source code, notebooks, and bug-bite dataset are maintained in the project GitHub repository.

Data sources The cyclone dataset is derived from:

- HURSAT-B1: https://www.ncei.noaa.gov/products/hurricane-satellite-data
- IBTrACS: https://www.ncei.noaa.gov/products/international-best-track-archive

Both are freely available from NOAA.

Model library Architectures are loaded via the timm library (https://github.com/huggingface/pytorch-image-models). Pre-trained ImageNet weights are downloaded automatically by timm on first use.